

DEEPSEARCHER STUDY

第二阶段学习资料

详细学习讲义 · 阶段面试问答复习手册

覆盖 FastAPI、产品工作台、结构化 Trace、React、RAG 评测、检索策略、性能、安全与系统设计

生成日期: 2026-07-26

目录

第一部分用于系统复习源码与工程链路；第二部分用于面试前快速回忆标准表达与诚实边界。

第一部分 第二阶段学习讲义	3
1. FastAPI 业务入口：把 HTTP 请求接到 DeepSearcher	3
2. 用户工作台与文件上传：产品链路和兼容链路	9
3. Trace 全栈链路：应用事件不是模型思维链	9
4. React 状态与测试：产品工作台与学习控制台并存	10
5. RAG 评测：把“看起来不错”变成可重复实验	10
6. 延迟、Token 与成本：分解而不是只看总数	11
7. Dense、BM25、Hybrid 与 RRF	11
8. 并发、安全与十倍流量	12
9. 生产系统设计与 SSE	12
10. 项目表达：结论、因果、证据、取舍、边界	13
11. 第二阶段不能说错的事实	14
第二部分 第二阶段面试问答复习手册	15
一、FastAPI 与后端数据流高频题	15
二、FastAPI 面试前十分钟速记	16
三、本地代理、前端与 Trace 高频题	16
四、RAG 评测、性能与检索高频题	17
五、并发、安全与生产系统设计高频题	18
六、项目表达与质疑型追问	20
七、第二阶段面试前十分钟速记	21

第一部分 第二阶段学习讲义

从业务入口出发，沿当前源码理解产品上传、Trace、React 状态、评测、性能、检索与生产架构。

范围：AI 全栈后端、可观测性、前端数据流、RAG 评测、并发、安全与系统设计。当前进度：第二阶段与第一遍详细学习已完成；下一步进入第二遍模拟面试。用途：承接第一阶段 RAG 与 Agent 基础，继续第一遍详细学习，避免单个阶段文档过长。前置材料：第一阶段学习讲义 · 第二阶段面试问答复习手册 · 项目问题与优化清单

1. FastAPI 业务入口：把 HTTP 请求接到 DeepSearcher

1.1 先记住一条中文链路

客户端 HTTP 请求
→ FastAPI 解析并校验参数
→ main.py 选择配置、入库或查询函数
→ online_query.py 读取当前全局 Searcher
→ RAGRouter 选择 DeepSearch 或 ChainOfRAG
→ LLM、Embedding、Milvus 完成查询
→ main.py 把结果或异常转换成 HTTP 响应

FastAPI 解决的是“怎样把 Python 业务能力安全、稳定地暴露成网络接口”，它本身不负责 RAG。`main.py` 是 HTTP 适配层，`deepsearcher/online_query.py` 是很薄的业务门面，Agent、Embedding 和 Milvus 才是查询主体。

1.2 应用启动时发生什么

`main.py` 先调用 `load_dotenv()`，让后续创建模型客户端时能够读取 `.env` 中的密钥和地址；然后创建 FastAPI 应用、读取 YAML 配置，并在模块导入阶段立即调用 `init_config(config)`。

`init_config()` 会同步创建：

- LLM 客户端。
- Embedding 客户端。
- 文件加载器和网页爬虫。
- VectorDB 客户端。
- 由 DeepSearch 和 ChainOfRAG 组成的默认 RAGRouter。
- 作为基础能力保留的 NaiveRAG。

这样设计对单进程本地演示很直接：只要导入应用，后面的接口就能使用全局组件。代价是模块导入与外部依赖初始化解绑在一起，测试仅仅 `import main` 也可能等待真实组件；启动失败也难以区分是应用代码、模型配置还是 Milvus 连接问题。更可控的替代方案是使用 FastAPI lifespan 在启动阶段创建资源，通过依赖注入把服务传给接口，并在关闭阶段释放连接。

当前还有两个容易忽略的事实：

1. `main.py` 有自己的 `config = Configuration()`, `deepsearcher.configuration` 模块也有一个同名配置对象；实际传给 `init_config()` 的是 `main.py` 的对象。
2. `/set-provider-config/` 只修改当前进程内存中的配置，没有把改动写回 `config.yaml`。重启后会恢复文件配置，多 Uvicorn worker 之间也不会自动同步。

1.3 四个业务接口分别做什么

接口	输入位置	当前作用	关键边界
<code>POST /set-provider-config/</code>	JSON Body	修改某类 provider 配置并重建全部全局组件	进程内生效；无鉴权；响应可能原样回显包含密钥的 <code>config</code>
<code>POST /load-files/</code>	JSON Body	读取后端机器上的文件路径，切 Chunk、Embedding 后写入 Milvus	不是浏览器二进制上传接口；默认 <code>batch_size=256</code>
<code>POST /load-website/</code>	JSON Body	抓取 URL、切 Chunk、Embedding 后写入 Milvus	依赖爬虫和外部网络；当前入口没有暴露抓取超时等治理参数
<code>GET /query/</code>	Query String	调用默认 RAGRouter 查询，可选返回 Trace	问题会出现在 URL；返回的是最终答案和 Token，不返回检索结果

Pydantic 模型 `ProviderConfigRequest` 会校验 `feature`、`provider` 和 `config` 三个字段是否存在以及基本类型。其他接口通过 `Body()` 和 `Query()` 声明参数。比如 `max_iter` 设置了 `ge=1`，传入 `0` 时请求会在进入业务函数之前被 FastAPI/Pydantic 拒绝，通常返回 422。

但“类型校验通过”不等于“业务输入安全”。例如：

- `feature` 只是普通 `str`，是否受支持要到 `set_provider_config()` 才判断。
- `paths` 没有目录白名单，能否读取完全取决于服务进程的文件权限。
- `collection_name` 没有在 API 层定义完整命名规则。
- `original_query` 没有长度、内容和成本预算限制。
- 所有业务接口都没有身份认证和授权。

`main.py` 还提供 `--enable-cors` 开关，但它只在直接执行 `python main.py`、进入 `if __name__ == "__main__"` 分支时添加 CORS 中间件；常见的 `uvicorn main:app` 启动方式不会执行这段代码。并且 `argparse` 使用 `type=bool`，字符串 `"False"` 在 Python 中仍会转换为真值。当前开启后又允许任意 origin、method 和 header，不应把它当成生产级跨域策略。更稳妥的做法是用 `store_true` 或配置项，并显式列出可信前端 origin。

1.4 一个查询请求怎样进入 DeepSearcher

假设客户端请求：

```
GET /query/?original_query=Milvus%20适合什么场景&max_iter=3&include_trace=true
```

完整执行顺序是：

1. FastAPI 把三个查询参数转换为 `str`、`int` 和 `bool`，并检查 `max_iter >= 1`。

2. `perform_query()` 看到 `include_trace=True`, 调用 `query_with_trace(original_query, max_iter)`。
3. `query_with_trace()` 创建 `TraceCollector`, 从 `deepsearcher.configuration.default_searcher` 读取当前 `RAGRouter`。
4. `RAGRouter` 先让 LLM 在 `DeepSearch` 与 `ChainOfRAG` 中选择一个 Agent。
5. 被选中的 Agent 做 `Collection` 路由、`Embedding`、`Milvus` 检索和答案生成, 并把受控应用事件写入 `Collector`。
6. `query_with_trace()` 返回 `(answer, retrieval_results, consume_tokens, trace)`。
7. `perform_query()` 丢弃第二项检索结果, 只向客户端返回 `result`、`consume_token` 和 `trace`。

未要求 `Trace` 时, 入口调用 `query()`, 响应保持原有结构:

```
{
  "result": ".....",
  "consume_token": 1234
}
```

`online_query.py` 没有在导入时复制 `default_searcher`, 而是在每次调用时读取 `configuration.default_searcher`。这使配置切换后新请求能拿到新对象, 但也带来并发代际问题: 请求 A 可能已经拿到旧 `Searcher`, 请求 B 重建全局对象后再拿到新 `Searcher`。单次调用中的局部变量仍指向旧对象, 不会凭空变成新对象, 但系统会暂时存在新旧两套客户端, 且切换过程没有锁、版本号 and 原子发布协议。

1.5 文件入库请求怎样执行

`POST /load-files/` 接收的是一个后端可见路径或路径列表, 而不是文件字节。入口把参数交给 `load_from_local_files()` 后, 执行:

```
路径存在性检查
→ 文件或目录解析为 Document
→ 按 1500/100 默认参数切 Chunk
→ 按 batch_size 批量生成 Embedding
→ 写入指定或默认 Collection
```

例如请求体:

```
{
  "paths": ["D:/knowledge/a.pdf", "D:/knowledge/b.pdf"],
  "collection_name": "interview_notes",
  "batch_size": 64
}
```

这里的路径必须存在于 `FastAPI` 服务所在机器。浏览器用户电脑上的 `C:/Users/.../a.pdf` 对部署在另一台服务器上的后端没有意义。这正是后续本地代理需要先接收浏览器文件、保存临时文件, 再把“后端可见路径”传给该接口的原因。

当前入口没有暴露 `chunk_size`、`chunk_overlap` 和 `force_new_collection`, 因此 API 入库固定使用底层函数默认的 `1500/100`, 并默认不删除 `Collection`。这减少了误操作入口, 但也降低了调参灵活性。

1.6 def 与 async def 在这里有什么区别

当前四个路由函数都是普通 `def`。在本项目锁定的 FastAPI 0.115.12 / Starlette 0.46.2 中，FastAPI 会在线程池中执行同步路由，避免它们直接占住主事件循环。

必须区分三句话：

1. `def` 会被放入线程池：其他异步连接仍有机会被事件循环处理。
2. 线程池不等于无限并发：一个长时间等待 LLM、Embedding 或 Milvus 的请求会持续占用一个工作线程；线程耗尽后新请求仍要排队。
3. 把签名改成 `async def` 不会自动变快：如果函数内部仍直接调用同步 `llm.chat()`、`embed_query()` 和 Milvus 客户端，反而会直接阻塞事件循环。

正确选择取决于依赖：

- 依赖只有同步客户端：保留 `def`，或显式放入容量受控的线程池，并配置超时与并发上限。
- 依赖提供真正异步客户端：使用 `async def`，每层都 `await`，并把取消和 `deadline` 向下传播。
- PDF 解析等 CPU 密集任务：线程受 Python GIL 和 CPU 竞争影响，更适合任务队列或进程池，而不是占用 Web 请求线程。
- 大批量入库：更适合提交后台任务并返回 task ID，由 Worker 完成解析、Embedding 和写库。

当前 DeepSearch 内部虽然出现 `asyncio.gather()`，但其中仍包含同步模型和向量库调用，所以不能把整个系统描述成“异步高并发架构”。

1.7 异常怎样变成 HTTP 响应

当前有两层异常行为：

1. 请求缺字段、类型错误或 `max_iter < 1`，由 FastAPI/Pydantic 在进入业务函数前返回 422。
2. 进入业务函数后发生的任何 `Exception`，都被入口捕获并转换成 `HTTPException(status_code=500, detail=str(e))`。

第二层实现简单，却不稳定：文件不存在、非法 feature、上游限流、模型鉴权失败、Milvus 不可用和代码 Bug 都会变成 500；同时 `str(e)` 可能泄漏内部路径、供应商响应或连接信息。`/set-provider-config/` 成功时还会把 `request.config` 原样返回，如果其中包含 `api_key`，密钥就会出现在响应、浏览器状态或代理日志里。

更稳健的映射应先建立错误分类，例如：

场景	建议状态	稳定错误码示例
参数语义不合法	400/422	INVALID_ARGUMENT
文件不存在	404	SOURCE_NOT_FOUND
Collection 状态冲突	409	COLLECTION_CONFLICT
上游模型限流	429 或受控 503	UPSTREAM_RATE_LIMITED

场景	建议状态	稳定错误码示例
LLM、Embedding、Milvus 暂时不可用	502/503	DEPENDENCY_UNAVAILABLE
请求超过 deadline	504	QUERY_TIMEOUT
未分类内部错误	500	INTERNAL_ERROR

客户端响应只返回稳定 `code`、安全 `message` 和 `request_id`；服务端日志记录完整堆栈和依赖错误，并对密钥、绝对路径和文档内容脱敏。还要注意，如果以后在当前 `try` 内主动抛出 `HTTPException`，宽泛的 `except Exception` 也可能把它重新包装成 500，因此应先 `except HTTPException: raise`，再处理领域异常和未知异常。

1.8 相关 API 测试实际保护了什么

当前 `tests/test_query_api.py` 有两个测试，通过 monkeypatch 替换 `main.query` 或 `main.query_with_trace`，再直接调用 `perform_query()`：

- 不请求 Trace 时，保护旧响应只有 `result` 和 `consume_token`。
- 请求 Trace 时，保护响应额外包含 `trace`。

它们能防止返回结构回归，但没有经过 ASGI、HTTP 参数解析和 Pydantic 校验，因此没有验证：

- URL 到函数参数的转换。
- `max_iter=0` 是否返回 422。
- 业务异常的状态码和安全响应。
- 配置、文件入库和网页入库接口。
- CORS、认证、超时、并发配置切换。
- 应用启动和依赖关闭。

而且测试文件在 monkeypatch 之前就 `import main`，会触发真实 `init_config()`。本次按当前工作区运行该测试时，测试在合理等待时间内没有完成，因此已终止，没有把它记成“测试通过”或“测试失败”。准确说法是：当前测试设计仍受启动期外部组件初始化影响，必须先解耦依赖，才能得到稳定的 API 单元测试。

合理的测试分层是：

1. 纯函数单元测试：直接测试错误分类和响应 DTO。
2. FastAPI `TestClient`/ASGI 测试：覆盖 200、422、404、503、504 和响应脱敏。
3. 集成测试：使用可控的测试 Milvus 与模型 Stub，验证完整调用链。
4. 并发测试：配置切换与查询同时发生时，验证版本隔离和资源回收。

1.9 为什么当前方案仍有价值，以及替代方案的代价

当前单文件入口适合学习和本地 Demo：代码少、调用链直、Swagger 文档可自动生成，也容易从函数跳进业务代码。若过早拆成很多 Controller、Service、Repository，反而会增加理解成本。

但当目标变为多人、多租户或生产服务时，需要付出结构化成本：

- lifespan 和依赖注入增加代码层次，但换来可测试的资源生命周期。
- 任务队列增加部署组件和状态管理，但避免大文件入库拖垮 Web 线程。
- 领域错误模型需要维护映射表，但客户端才能可靠重试和提示。
- 配置中心或版本化配置增加基础设施，但多 worker 和多实例才能一致。
- 认证、路径白名单与密钥脱敏增加开发工作，却是公开服务的基本安全边界。

1.10 常见错误说法

错误说法	正确说法
FastAPI 接口是 <code>def</code> ，所以一定阻塞整个服务	同步路由通常在线程池执行，不直接阻塞事件循环，但会占用有限工作线程
改成 <code>async def</code> 就能提高并发	只有整条依赖链真正异步并正确 <code>await</code> 才可能改善并发
<code>/load-files/</code> 是文件上传接口	它接收后端文件系统路径，不接收文件字节
配置接口修改了 YAML	它只修改当前进程内存并重建全局对象
所有错误返回 500 很统一	这会混淆客户端错误、依赖故障和服务 Bug，也不利于重试
有两个 monkeypatch 测试就证明 API 可用	当前测试只覆盖函数返回形状，没有覆盖真实 HTTP 和依赖生命周期
使用多个 Uvicorn worker 就能共享新配置	每个 worker 是独立进程，内存配置不会自动同步

1.11 面试标准表达

DeepSearcher 的 FastAPI 入口是 HTTP 适配层。应用当前在导入 `main.py` 时读取配置并初始化 LLM、Embedding、Milvus 和默认 RAGRouter。查询接口经过参数校验后调用 `online_query.py`，后者读取当前全局 Searcher，由 RAGRouter 在 DeepSearch 与 ChainOfRAG 之间选择，再返回答案、Token 和可选的结构化 Trace。四个业务路由目前都是同步 `def`，FastAPI 会把它们放在线程池执行，但同步模型和向量库调用仍会占用有限线程；简单改成 `async def` 不能解决问题。当前实现适合单用户本地演示，生产化还需要依赖注入和生命周期管理、版本化配置、认证授权、超时限流、后台入库任务，以及稳定且脱敏的领域错误响应。

1.12 当前项目的诚实边界

- 已有可读的配置、文件入库、网页入库、查询和可选 Trace 接口。
- 已有 FastAPI/Pydantic 基础参数校验，但没有完整业务校验、认证与授权。
- 当前是同步阻塞依赖加线程池承载，不是端到端异步架构。
- 配置切换是进程内全局替换，没有并发版本隔离，也没有多 worker 同步。
- 文件接口读取服务端路径，不是可直接公开给远程浏览器的安全上传方案。
- 业务异常目前统一变成带原始错误文本的 500，尚未实现稳定错误协议和脱敏。
- 当前 API 测试只保护查询返回结构，而且仍与导入期真实依赖初始化耦合。

- CORS 只在直接运行脚本时按命令行参数安装，启动方式不同会得到不同策略，且当前策略过于宽泛。
- 没有负载测试数据，不能声称当前接口已经达到生产级吞吐、可用性或安全性。

2. 用户工作台与文件上传：产品链路和兼容链路

2.1 当前产品工作台链路

```
浏览器选择 PDF
→ product-api.ts 用 FormData 发送 multipart
→ /api/knowledge-bases/{id}/documents 读取并校验
→ 计算 SHA-256，拒绝知识库内重复 PDF
→ 保存 Document、IngestJob 和持久文件，状态为 queued
→ 返回 202，BackgroundTasks 将状态改为 processing
→ 用服务端文件路径调用 DeepSearcher /load-files/
→ 成功变为 ready，失败变为 failed
→ React 在 queued/processing 时每两秒轮询
```

`product-api.ts` 不手工设置 multipart 的 `Content-Type`，而是让浏览器根据 FormData 生成带 boundary 的 Header。服务端使用文件名、Content-Type、20 MiB、`%PDF` 魔数和 SHA-256 做基础校验；`Path(display_name).name` 去掉目录部分，实际存储名使用内部 Document ID。

当前路由仍通过 `await file.read()` 把文件完整读入内存后才校验大小；BackgroundTasks 也运行在 Web 进程内。因此 multipart 消除了 Base64 约三分之一的传输膨胀，却没有解决大请求内存、恶意 PDF 隔离、进程崩溃恢复或多 Worker 调度。生产系统更适合网关限流、对象存储直传和持久化任务队列。

2.2 `/console` 旧学习控制台兼容链路

```
浏览器选择 PDF
→ api.js 转成 Base64 JSON
→ /api/ingest 校验并写入 TemporaryDirectory
→ 把临时服务端路径交给 /load-files/
→ DeepSearcher 在请求返回前读完
→ 请求结束后自动清理临时目录
```

这条链路仍有学习价值：它解释了浏览器文件与服务端路径的边界，也展示了为什么 BFF 能适配旧接口。但它不再是默认产品页面的上传方式。面试时必须明确“产品页是 multipart + 持久状态，`/console` 才是 Base64 + 临时目录”。

3. Trace 全栈链路：应用事件不是模型思维链

3.1 数据怎样流动

```

query_with_trace()
→ 创建请求级 TraceCollector
→ RAGRouter 记录 Agent
→ ChainOfRAG 逐轮记录子问题、Collection、检索文档、
  中间答案、支持文档、反思和 Token
→ build() 生成版本化安全 JSON
→ 学习控制台: FastAPI → /api/query → api.js → /console 的 TracePanel
→ 产品工作台: Trace → Citation 持久化 → product-api.ts → 引用抽屉

```

Trace 记录程序显式可见的输入、输出和阶段事件，不读取也不声称暴露模型隐藏思维链。当前脱敏策略包括：最多展示 5 条文档、每条 600 字，不返回 Embedding 和原始 metadata；本地路径只保留文件名，URL 删除查询参数，引用 metadata 使用白名单。

当前 Trace 主要完整覆盖 **ChainOfRAG**。路由到 **DeepSearch** 时可能只有 Agent 和总 Token，没有等价的逐轮事件。**build()** 在整个查询结束后才执行，因此页面显示的是完成后回放，不是实时 Trace。

4. React 状态与测试：产品工作台与学习控制台并存

当前主入口已经是 TypeScript 产品工作台 **App.tsx**，由 React Router 提供四组产品路由：

```

/           新对话
/chat/:conversationId 持久化对话与引用抽屉
/knowledge      知识库列表
/knowledge/:id    文档上传、状态轮询和失败重试
/console        原学习控制台

```

产品工作台使用 TanStack Query：**useQuery** 管理知识库、文档和对话缓存，**useMutation** 管理创建、上传、提问与重试，成功后通过 **invalidateQueries** 让服务端状态重新成为事实来源。文档处于 **queued/processing** 时每两秒轮询，至少有一份 **ready** 文档后才允许提问。问答仍由 **sendMessage()** 等待完整 JSON，再刷新 Conversation；当前没有 SSE 或 **AbortController**。

旧学习控制台保留在 **/console**。它仍通过 **api.js** 调用 **/api/query**，一次写入答案、延迟、Token 和 Trace，并用 **TracePanel** 回放应用事件。因此面试时必须区分“产品主页面”和“调试学习控制台”，不能再把两者统称为一个页面。

现有组件测试保护默认工作台、创建对话、恢复回答、重新生成、本地反馈、引用抽屉焦点恢复和 **/console** 回退入口；API 与 Trace 测试保护旧控制台的字段映射和展示。它们不能证明真实 LLM、Milvus、浏览器布局、并发竞态或完整无障碍行为。

5. RAG 评测：把“看起来不错”变成可重复实验

完整评测需要分三层：

```

检索层：金标证据是否进入 TopK
生成层：给定证据后答案是否正确、忠实、可引用
系统层：延迟、Token、成本、失败率和稳定性

```

仓库自带 `evaluation/evaluate.py`，使用 2WikiMultiHopQA 比较 DeepSearcher 与 Naive RAG 的 `Recall@2/Recall@5`，并统计 DeepSearcher Token 和流水线错误。它从 `supporting_facts` 中提取文档标题，因此主要是标题级检索评测；脚本没有使用标准答案字段，也就没有评最终答案正确性、Faithfulness、引用正确性和拒答能力。

第一版业务黄金集可以先做 20 题，覆盖单跳、多跳、比较、精确实体、不可回答和冲突问题。每题至少保存：

```
question
reference_answer
answerable
expected_sources / chunk_ids
question_type
corpus_version
```

常用检索指标包括 Recall@K、MRR、nDCG、重复率和空结果率；生成指标包括 Answer Correctness、Faithfulness、Citation Correctness 和拒答准确率。诊断时先判断金标证据是否进入 TopK：未进入是检索问题；进入后仍答错，主要检查生成和上下文组织。

6. 延迟、Token 与成本：分解而不是只看总数

代理通过 `perf_counter()` 测量“代理准备调用上游到完整响应解析完成”的近似端到端延迟。它包含 FastAPI、LLM、Embedding 和 Milvus 等等待，却不包含完整浏览器网络和 React 渲染，也没有各阶段 Span。

平均延迟会被大量快请求稀释，生产系统至少报告 p50、p95、p99、超时率和成功率。180 秒代理超时只是失败边界，不是 SLO。

当前 Token 来自每次 `ChatResponse.total_tokens` 的手工累加：包括 Agent 路由、Collection 路由、子问题、中间答案、支持文档筛选、可选反思和最终回答。它没有拆分 input/output/cached/reasoning Token，也没有统计 Embedding 使用量，所以只能近似估算费用。

ChainOfRAG 在单 Collection、`max_iter=3` 的常见路径中大约需要一次 Agent 路由、每轮三次 LLM 调用和一次最终回答，即约 11 次调用。后续轮次还会携带累积上下文，因此 Token 和延迟不保证严格线性增长。优化目标应是质量、p95、Token、失败率的 Pareto 折中，而不是只追求最高正确率。

7. Dense、BM25、Hybrid 与 RRF

```
Dense: 根据 Embedding 找语义近似
BM25: 根据词频、稀有度和文档长度找字面匹配
Hybrid: 同时执行两路召回
RRF: 按名次而不是原始分数融合
```

Dense 擅长同义词和自然语言改写，但对错误码、SKU、函数名和版本号可能不稳定；BM25 擅长稀有精确字符串，却不能普遍理解同义表达。RRF 使用 $\sum 1/(k + \text{rank})$ 融合排名，避免直接比较 L2 与 BM25 这两种方向、范围和量纲都不同的分数。

Milvus 实现已经具备 Analyzer、`sparse_vector`、BM25 Function、稀疏倒排索引、双路 `AnnSearchRequest` 和 `RRFRanker()`。但当前 YAML 没有启用 `hybrid`，实际仍是 Dense-only。已有 Collection 也不能只靠开关升级：它缺少稀疏字段、Function 和索引，需要新建 Schema 并重新入库。

当前 `RetrievalResult.score` 承接 Milvus 的 `distance`，默认 L2 越小越近，而前端笼统标为“相似度”不够严谨。生产响应应同时携带 `metric/score_type/higher_is_better/retrieval_mode`。

8. 并发、安全与十倍流量

并发流量会被 Agent 放大。10 个用户请求在 ChainOfRAG 下可能形成约百次 LLM 调用需求，真正瓶颈可能是线程池、RPM/TPM、Milvus、内存和排队，而不只是 FastAPI。

同步 `def` 路由通常由 FastAPI 放入线程池，不直接阻塞事件循环，但每个长查询会长期占用有限线程。把签名改成 `async def` 不能把同步 LLM、Embedding 和 Milvus SDK 自动变成非阻塞。

模块级共享客户端不一定错误：只读、无请求状态且支持并发的客户端可以复用连接。高风险在于 `/set-provider-config/` 逐个替换全局对象且没有原子版本；多 worker 又各自持有独立内存，配置会不一致。

当前高优先级安全边界包括：

1. 后端接口没有鉴权和限流。
2. `/load-files/` 接受服务器路径，直接公开可能读取任意可访问文件。
3. 调用方可指定 `collection_names`，不能作为多租户授权。
4. 代理限制 `max_iter ≤ 10`，但直接后端没有上限，校验可被绕过。
5. `/set-provider-config/` 属于控制面，还可能回显请求中的密钥。
6. URL 抓取可能产生费用滥用、SSRF 和恶意内容入库风险。
7. GET 查询把问题放入 URL，可能进入访问日志。

十倍流量的改造顺序应是：入口鉴权限流和请求体上限 → 有界并发与背压 → 入库任务队列 → 外部依赖超时、预算和熔断 → 租户授权 → 最后再水平扩容。单纯增加 Uvicorn worker 不能提高模型限额，还会复制连接和进程内配置。

9. 生产系统设计与 SSE

当前 `query_with_trace()`、代理 `response.json()` 和浏览器 `response.json()` 都要等待完整响应。升级路线应先定义事件协议，再改传输：

```
POST /query-runs
→ 鉴权、参数校验、Idempotency-Key
→ 创建 run/message/outbox
→ Query Worker 执行 Agent
→ 事件先写 Event Store，再发布
→ GET /query-runs/{id}/events 使用 SSE
→ DELETE /query-runs/{id} 显式请求取消
```

建议事件包括 `run.started`、`routing.completed`、`iteration.started`、`subquery.generated`、`retrieval.completed`、`support_filter.completed`、可选 `reflection.completed`、`final_answer.completed`，以及唯一终态 `run.completed/run.failed/run.cancelled`。

每个事件携带 `schema_version/run_id/trace_id/event_id/timestamp/payload`。SSE 重连可能重放事件，前端必须按 `event_id` 幂等归并；答案 Delta 需要 `chunk_index` 或 `offset`，不能盲目字符串追加。

客户端断线不等于任务取消。持久化会话通常让任务继续，刷新后按 `Last-Event-ID` 恢复；用户点击停止才发送取消请求。同步 LLM 调用不一定能立即中断，所以要区分 `cancellation_requested` 和 `cancelled`。

当前主页面已经接入 KnowledgeBase、Document、IngestJob、Conversation、Message 和 Citation 的持久状态模型。产品上传使用 multipart，FastAPI 返回 202 并通过 `BackgroundTasks` 处理；旧 `/console` 上传才继续使用 Base64 适配。BackgroundTasks 仍在 Web 进程内执行，不是具备租约、死信、进程恢复和多 Worker 调度的持久任务队列；Conversation 问答也仍等待完整 JSON。

10. 项目表达：结论、因果、证据、取舍、边界

面试表达固定使用五步：

先给结论
→ 讲因果链
→ 给当前源码或测试证据
→ 说明为什么不选其他方案
→ 主动说尚未实现和下一步

不要堆砌技术名词，也不要路线写成成果。能够展示的证据优先级是：可运行结果和指标 → 测试输出 → 源码调用链 → 设计文档。没有评测数字时直接说明没有，不编造“准确率提升 30%”。

10.1 当前可复述的一分钟版本

DeepSearcher 是一个把私有文档向量化后进行多策略 RAG 问答的项目。离线侧由 PDFLoader 按页解析，按 1500/100 默认参数切成 Chunk，用千问 `text-embedding-v4` 生成 1024 维向量并写入 Docker Milvus；在线侧先由 RAGRouter 在 DeepSearch 与 ChainOfRAG 中选择，再完成 Collection 路由、迭代检索和 `qwen-plus` 最终回答。当前工作区增加了 FastAPI 用户工作台、知识库和持久化对话、PDF 任务状态、可核对引用，以及保留在 `/console` 的学习控制台和脱敏结构化 Trace。它已经具备本地学习和定向验证基础，但仍没有正式业务黄金集、SSE、鉴权、限流和生产级任务队列。

10.2 真实验证基线

2026-07-26 按当前工作区运行：

```
uv run --frozen pytest \
  tests/test_trace.py \
  frontend/tests/test_server.py \
  frontend/tests/test_product_api.py \
  frontend/tests/test_product_data.py \
  tests/test_p0_knowledge_citations.py -q
```

32 passed in 4.83s

前端:

```
npm test
```

```
Test Files 3 passed (3)
Tests      10 passed (10)
Duration   5.49s
```

同一版本的 `npm run build` 也成功完成，Vite 转换了 583 个模块。包含 `tests/test_query_api.py` 的 Python 组合在测试收集阶段导入 `main.py`，随即连接 `127.0.0.1:19530`；由于本次 Milvus 不可用，得到 1 个收集错误，测试用例尚未执行。准确说法是“依赖隔离后的 32 个 Python 定向测试、10 个前端测试和前端生产构建通过；入口测试受导入期 Milvus 初始化阻塞”，不能说全量测试全绿。

11. 第二阶段不能说错的事实

1. 本地代理上传的是文件内容，DeepSearcher `/load-files/` 接收的是后端路径。
2. Trace 是显式应用事件，不是模型隐藏思维链。
3. 当前 Trace 完成后一次性返回，不是 SSE。
4. `latency_ms` 是代理等待后端的近似端到端时间，没有阶段 Span。
5. `consume_token` 主要是手工累计的 LLM 总 Token，不含完整 Embedding 计费。
6. 仓库有 2WikiMultiHopQA 检索 Recall 脚本，但没有当前业务语料的正式完整评测。
7. 当前配置是 Dense-only；Hybrid 只是已有代码分支。
8. 默认 L2 是距离，越小越近，不能无条件称为相似度。
9. 同步路由在线程池中运行，不等于端到端异步或无限并发。
10. 全局共享客户端可以合理，但全局热更新和多 worker 配置不一致是风险。
11. 代理校验不是最终安全边界，直接后端必须重复校验与授权。
12. `BackgroundTasks` 不是持久化分布式任务队列。
13. 客户端断线不等于任务取消，SSE 也不保证 Exactly Once。
14. 当前产品状态和引用基础不能证明答案所有声明都 grounded。
15. 没有正式压测和质量指标，不能声称生产级高并发或准确率提升。
16. 当前默认页面是产品工作台；旧学习控制台只保留在 `/console`。
17. 产品上传使用 multipart；Base64 上传属于旧学习控制台接口。

第二部分 第二阶段面试问答复习手册

每题按结论、深入回答与诚实边界组织。复习时先口述，再对照标准答案。

范围：AI 全栈后端、可观测性、前端数据流、评测、并发、安全与系统设计。当前进度：第二阶段已完成；本册覆盖后端、全栈 Trace、评测、性能、安全和系统设计。原则：只记录真实高频主问题和合理追问，不把规划中的生产能力说成已经实现。配套材料：第二阶段学习讲义·项目问题与优化清单

一、FastAPI 与后端数据流高频题

1. 一次查询请求如何从 FastAPI 进入 DeepSearcher?

标准回答：

GET /query/ 先由 FastAPI/Pydantic 解析 `original_query`、`max_iter` 和 `include_trace`。入口随后调用 `online_query.query()` 或 `query_with_trace()`，从 `deepsearcher.configuration` 读取当前全局 RAGRouter。RAGRouter 先选择 DeepSearch 或 ChainOfRAG，被选中的 Agent 再完成 Collection 路由、Embedding、Milvus 检索和答案生成。入口最终只返回答案、Token 和可选 Trace，不返回检索结果列表。

2. FastAPI 的 `def` 和 `async def` 有什么区别？当前项目属于哪一种？

标准回答：

同步 `def` 路由通常由 FastAPI 在线程池执行，不会直接占住事件循环；`async def` 在事件循环上执行，适合内部使用真正异步并可 `await` 的 I/O。当前四个业务接口都是 `def`，底层 LLM、Embedding 和 Milvus 调用也是同步的。它能用线程处理多个请求，但每个长调用都会占用有限工作线程。只把签名改成 `async def` 反而可能直接阻塞事件循环，必须逐层异步化或使用受控线程池、超时和并发上限。

3. /load-files/ 为什么不能直接理解为浏览器文件上传？

标准回答：

它接收的是 JSON 中的后端文件系统路径，随后由服务进程读取该路径。远程浏览器本机路径对服务器不可见，接口也没有接收 multipart 文件字节。因此产品工作台在外层提供 multipart 接口，持久化文件和任务状态后，再把服务端路径交给该入口；旧 `/console` 才使用临时目录兼容。生产中的大文件仍更适合对象存储直传加异步 Worker。

4. 当前配置切换为什么不适合并发和多 worker？

标准回答：

`/set-provider-config/` 修改当前进程内存中的 Configuration，并调用 `init_config()` 重建模块级 LLM、Embedding、VectorDB 和 Agent。它没有锁、配置版本和资源回收协议，并发请求可能暂时使用不同代际对象；

多个 Uvicorn worker 又是独立进程，内存状态不会自动同步。生产方案应使用版本化配置发布、依赖注入和请求级上下文，让旧请求自然结束后再回收旧实例。

5. 为什么不能把所有业务异常都返回 500?

标准回答：

500 应表示未预期的服务内部错误。文件不存在、参数语义错误、上游限流、依赖不可用和超时需要不同状态及稳定错误码，否则客户端无法判断是修改请求、重试还是稍后再试。当前入口还直接返回 `str(e)`，可能泄漏路径和上游信息。合理方案是领域异常分类，向客户端返回稳定 `code`、安全 `message` 和 `request_id`，完整堆栈只进入脱敏后的服务端日志。

6. 当前 FastAPI 测试覆盖到了什么，没覆盖什么？

标准回答：

现有两个测试 monkeypatch 查询函数后直接调用 `perform_query()`，主要保护有无 Trace 时的响应结构。它们没有经过真实 ASGI/HTTP 参数解析，没有覆盖 422、错误映射、其他三个接口、认证、超时和并发。测试还会在 monkeypatch 前导入 `main.py`，触发真实组件初始化。后续应把资源创建移入 lifespan 和依赖注入，再增加 TestClient、依赖 Stub、集成测试及并发测试。

诚实边界：本次当前工作区测试在导入初始化阶段未于合理时间内完成，已主动终止，不能称为通过或失败。

二、FastAPI 面试前十分钟速记

1. 同步 `def` 路由在线程池运行，但不代表端到端异步或无限并发。
2. `/load-files/` 接收后端路径，不是浏览器文件字节。
3. 配置切换只改进程内全局状态，多 worker 不会自动同步。
4. 参数校验错误可由框架返回 422，业务故障不应全部包装成 500。

三、本地代理、前端与 Trace 高频题

7. 为什么浏览器和 DeepSearcher 之间需要本地代理？

结论：代理把浏览器文件与同源 HTTP 请求转换成 DeepSearcher 当前能理解的后端路径和接口格式，同时统一超时与错误。

深入回答：浏览器不能把本机路径交给另一个服务读取。当前产品工作台使用 multipart 把 PDF 发给知识库文档接口，服务端持久化文件和任务状态后返回 202，再将服务端路径交给 `/load-files/`。保留在 `/console` 的旧学习控制台仍使用 Base64、临时目录和 `/api/ingest` 适配。查询侧也由产品会话接口与旧 `/api/query` 分别服务两套界面。

诚实边界：multipart 已解决 Base64 放大，但当前产品上传仍依赖 Web 进程本地文件和 BackgroundTasks。生产环境更适合对象存储直传加持持久化 Worker；代理也不是鉴权和校验的最终安全边界。

8. Trace 为什么不是模型思维链？

结论：Trace 是程序显式记录的路由、检索和 Token 等应用事件，没有读取模型隐藏推理。

深入回答：TraceCollector 只接收代码已经获得的 Agent 名称、子问题、Collection、检索结果、中间答案、支持文档、反思布尔值和 Token。它最多展示五条文档、每条六百字，清理路径和 URL 参数，并对白名单 metadata 序列化。

诚实边界：当前逐轮事件主要覆盖 ChainOfRAG；DeepSearch 可能只有 Agent 和汇总。Trace 也是查询结束后一次性 build()，不是实时 SSE。

9. 当前产品工作台怎样管理 React 状态？旧控制台还有什么风险？

结论：产品工作台使用 TanStack Query 管理服务端状态和 Mutation，旧控制台仍使用本地 useState；两者当前都等待一次性 JSON。

深入回答：产品工作台按知识库、文档、会话设置 Query Key，Mutation 成功后使相关缓存失效并重新读取服务端；文档处理期间每两秒轮询，按钮通过 isPending 防止同一表单重复提交。旧 /console 的 api.js 和多个 setState 仍可能在并发请求下出现旧响应覆盖新响应。流式化后仍需事件状态机、run_id、取消和幂等归并。

诚实边界：现有组件测试保护产品默认页、对话恢复、重新生成、引用抽屉焦点和控制台回退入口，不证明真实浏览器布局、网络取消或完整无障碍行为。

四、RAG 评测、性能与检索高频题

10. 你会怎样评估一个 RAG 系统？

结论：分检索、生成和系统三层评估，不能用一个总分掩盖失败位置。

深入回答：检索层看 Recall@K、MRR、nDCG、重复率和空结果率；生成层看答案正确性、Faithfulness、引用正确性和拒答准确率；系统层看 p50/p95/p99、Token、成本、失败率。黄金集需要问题、标准答案、可回答标记和金标文档或 Chunk，并绑定语料、Chunk、Embedding、模型和 Prompt 版本。

诚实边界：仓库已有 2WikiMultiHopQA 标题级 Recall 评测，但没有当前业务语料的完整答案与忠实性基线。

11. 召回好但回答差，和召回差但答案像真的，怎样区分？

结论：先检查金标证据是否进入最终 TopK，再评价答案和证据关系。

深入回答：金标进入 TopK 但答案错，主要检查生成模型、Prompt、上下文组织和冲突处理；金标未进入但答案正确，模型可能依赖参数记忆，不能视为可靠 RAG 成功；未召回且模型拒答，检索失败但安全策略正常。

诚实边界：答案语义相似不等于事实正确，LLM Judge 也有偏差和非确定性，需要规则指标与人工抽查。

12. 为什么平均延迟不足以描述用户体验？

结论：平均值会被大量快请求稀释，无法描述外部模型和多轮 Agent 造成的长尾。

深入回答：应同时报告 p50、p95、p99、超时率和成功率，并把路由、Embedding、Milvus、每轮生成和最终回答拆成 Span。180 秒是超时边界，不是 SLO。质量、延迟、Token 和失败率要共同寻找 Pareto 折中。

诚实边界：当前只在代理层记录总体 `latency_ms`，没有阶段耗时和正式 SLO。

13. 当前 Token 统计能否直接计算精确成本？

结论：不能，它主要是各次 LLM `total_tokens` 的手工累计。

深入回答：当前包含 Agent 路由、子问题、中间回答、支持文档筛选和最终答案等 LLM 调用，但没有统一拆分 input/output/cached/reasoning Token，也没有统计 Embedding 使用量。精确成本还要结合模型分项单价、失败重试、Milvus 和基础设施成本。

诚实边界：不同 Provider 的 usage 语义可能不同，现有总 Token 更适合趋势比较，不应伪装成精确账单。

14. 为什么要混合 Dense 与 BM25？RRF 有什么作用？

结论：Dense 擅长语义，BM25 擅长错误码和专有名词等字面匹配；RRF 用排名融合，避免直接比较不兼容的原始分数。

深入回答：L2 越小越近，而 BM25 通常越大越相关，范围和量纲也不同。RRF 用 $\frac{1}{\sum 1/(k+rank)}$ 合并两路名次。Milvus 代码已经有 Analyzer、BM25 Function、稀疏倒排索引和 `RRFRanker()`。

诚实边界：当前配置仍是 Dense-only，已有 Collection 也需要重建 Schema 和重新入库才能启用 Hybrid；没有当前中文语料的收益报告。

五、并发、安全与生产系统设计高频题

15. 流量扩大十倍时为什么不能只增加 Uvicorn worker？

结论：Worker 只能增加进程级处理能力，不能提高 LLM RPM/TPM、Milvus 容量或保证全局配置一致。

深入回答：一个 ChainOfRAG 请求可能放大成约十次 LLM 调用。多 worker 会复制客户端、连接和进程内配置，`/set-provider-config/` 只修改命中的进程。正确顺序是入口鉴权限流、有界并发和背压、入库任务队列、依赖预算与熔断、租户隔离，最后再水平扩容。

诚实边界：当前没有正式并发压测数据，不能声称十倍流量下的吞吐或 p95。

16. 全局单例为什么不一定错误，当前真正的问题是什么？

结论：只读、无请求状态并支持并发的客户端可以共享；风险在非原子热更新、生命周期和多进程一致性。

深入回答：Trace 和 Agent 中间状态基本是请求局部变量，共享客户端还能复用连接。`init_config()` 却逐个替换 LLM、Embedding、VectorDB 和 Agent，没有版本或锁；失败时可能处于半更新状态，多 worker 还各自持有不同配置。

诚实边界：当前没有对 OpenAI-compatible 与 Milvus Client 的并发安全做专项验证，也没有原子 RuntimeBundle 和旧实例回收协议。

17. 当前最严重的安全边界有哪些？

结论：无鉴权的数据面和控制面、服务器路径入库、调用方指定 Collection、可绕过代理的参数上限，是高优先级风险。

深入回答：直接公开 `/load-files/` 可能读取进程可访问的任意路径；多租户下用户不能自行决定 Collection；直接后端可绕过代理的 `max_iter ≤ 10`；Provider 配置可能回显密钥；URL 抓取存在费用滥用、SSRF 和恶意文档风险；GET 查询可能把问题写入日志。

诚实边界：CORS 不是鉴权，Prompt 中写“不要泄露”也不能代替服务端文档授权。

18. 当前 Trace 非流式，怎样升级为 SSE？

结论：把 Collector 升级为事件发布者，后端、代理和前端逐事件传递，同时设计持久化、重连、幂等和取消。

深入回答：推荐 POST 创建 Query Run 并使用 Idempotency-Key，GET 订阅 `text/event-stream`，DELETE 请求取消。事件包含 `schema_version/run_id/trace_id/event_id`，类型包括 routing、iteration、retrieval、reflection、final 和唯一终态。事件先持久化再发布，客户端用 Last-Event-ID 重放并按事件 ID 去重。

诚实边界：当前没有 SSE、事件存储、AbortController 或可靠取消。SSE 改善首事件和感知延迟，不自动降低总耗时和成本。

19. SSE、WebSocket 和轮询为什么此处优先 SSE？

结论：当前主要是服务端单向推送 Agent 进度，SSE 基于 HTTP、协议简单且支持事件 ID；取消可以走独立 HTTP。

深入回答：WebSocket 更适合持续双向控制、实时协作和语音；轮询最兼容但有额外请求和进度延迟。原生 EventSource 不方便携带 POST Body 和 Bearer Header，因此可用“两步式 Run API + 同源 Cookie”，或使用 Fetch Stream。

诚实边界：SSE 长连接仍需要心跳、禁用代理缓冲、慢消费者控制、连接上限和权限校验。

20. FastAPI BackgroundTasks 是否已经解决可靠入库？

结论：没有，它改善了请求体验，但不是持久化分布式任务队列。

深入回答：当前产品 API 可以先保存 Document/IngestJob 状态并返回 202，再在 Web 进程执行 `process_document`。进程崩溃、部署重启或多副本场景下，没有任务租约、自动恢复、死信和统一 Worker 调度。生产应使用持久队列、幂等任务、重试预算和卡死任务回收。

诚实边界：当前已经有 `queued/processing/succeeded/failed` 状态基础，但不能描述为生产级异步任务平台。

六、项目表达与质疑型追问

21. 请用一分钟介绍项目

标准回答：

DeepSearcher 是一个把私有文档向量化后进行多策略 RAG 问答的项目。离线侧由 PDFLoader 按页解析，按 1500/100 默认参数切成 Chunk，用干问 `text-embedding-v4` 生成 1024 维向量并写入 Docker Milvus；在线侧先由 RAGRouter 在 DeepSearch 与 ChainOfRAG 中选择，再进行 Collection 路由、迭代检索和 `qwen-plus` 最终回答。当前工作区增加了 FastAPI 产品工作台，支持独立知识库、PDF 任务状态、持久化对话和可核对引用，并把带脱敏 Trace 的学习控制台保留在 `/console`。当前仍缺少正式业务黄金集、SSE、鉴权、限流和生产级任务队列，所以我把它定位为已跑通并持续产品化的学习项目，而不是成熟生产平台。

22. 你怎样证明项目真的跑过，而不是只读了源码？

结论：展示可复现命令、真实测试输出、一次端到端演示和明确的失败记录，不使用无法复现的数字。

当前证据：2026-07-26，不依赖实时 Milvus 的 Python 定向测试 `32 passed in 4.83s`；前端产品页更新后 3 个测试文件、10 个测试通过，`npm run build` 也成功完成。包含 `tests/test_query_api.py` 的组合在收集阶段因本机 Milvus 19530 不可用得到 1 个初始化错误，测试用例未执行。

诚实边界：这些结果不证明真实模型质量、Milvus 在线集成、并发能力或生产可用性。

23. 这个项目是不是包装上游项目？你的贡献是什么？

结论：先明确上游已有 RAG、Agent 和 Provider 抽象，再说明自己实际完成并能展示的全栈、可观测性和产品化改造。

深入回答：不能把 DeepSearch、ChainOfRAG 或 Milvus Provider 全部说成个人原创。当前可展示的改造集中在本地运行基线、版本化脱敏 Trace、产品工作台、知识库与会话持久化、文档任务状态、引用抽屉、保留的学习控制台及前后端契约测试。每项贡献都应给出源码、测试或演示证据。

诚实边界：如果某项只是路线图或当前工作区尚未稳定完成，就使用“正在实现/下一步”，不写成已上线成果。

24. 为什么不能说完整复现了 CoRAG、实现了高并发生产平台或准确率提升？

结论：这三种说法都缺少必要证据。

深入回答：当前 ChainOfRAG 只受 CoRAG 逐步检索思想启发，没有论文训练、拒绝采样等完整流程；项目没有鉴权、限流、正式 SLO 和压测，不能称高并发生产平台；仓库虽有公开数据集 Recall 脚本，但没有当前业务黄金集和改造前后报告，不能声称准确率提升百分比。

诚实边界：准确表达边界不会减分，反而说明能够区分实现、验证和规划。

七、第二阶段面试前十分钟速记

1. 回答顺序：结论 → 因果链 → 源码/测试证据 → 取舍 → 诚实边界。
2. Trace 是应用事件，当前完成后一次返回，不是思维链或 SSE。
3. 评测必须分检索、生成和系统三层。
4. p95/p99 比平均值更能描述长尾；超时不是 SLO。
5. Dense 找语义，BM25 找字面，RRF 按排名融合；当前未开启 Hybrid。
6. Token 总数不等于精确成本，当前也不含完整 Embedding usage。
7. `async` 关键字不能自动消除同步阻塞。
8. 全局共享客户端可以合理，但全局热更新和多 worker 一致性是风险。
9. 代理校验不是最终安全边界，Collection 名称也不是权限凭证。
10. BackgroundTasks 不是持久任务队列。
11. SSE 不保证 Exactly Once，断线也不等于取消。
12. 当前真实测试口径是“32 个 Python 定向测试和 10 个前端测试通过；Milvus 入口组合在收集阶段失败”，不是全量全绿。